



TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA TOÁN - CƠ - TIN HỌC

**Tìm hiểu các phương pháp giải chính xác cho bài toán định tuyến
phương tiện có ràng buộc khung thời gian (VRPTW)**

Người thực hiện:

Họ và tên	Mã sinh viên
Trần Đức Kiên	23000131
Tạ Văn Nghĩa	23000145
Vũ Tiến Đạt	23000111

Ngày 11 tháng 5 năm 2026

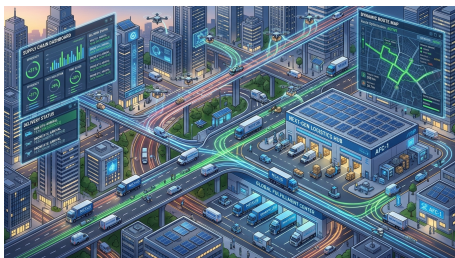
- 1 GIỚI THIỆU CHUNG
- 2 MÔ TẢ BÀI TOÁN
- 3 MÔ HÌNH HÓA BÀI TOÁN
- 4 BRANCH AND BOUND
- 5 BRANCH AND CUT
- 6 BRANCH AND PRICE
- 7 KẾT QUẢ THỰC NGHIỆM
- 8 KẾT LUẬN

1. GIỚI THIỆU CHUNG

Bối cảnh và đặt vấn đề

Bối cảnh

Trong bối cảnh ngành logistics và vận tải ngày càng phát triển, nhu cầu tối ưu hóa việc giao nhận hàng hóa trở nên đặc biệt quan trọng nhằm giảm chi phí vận hành, tiết kiệm thời gian và nâng cao chất lượng dịch vụ.



Hình: Tối ưu hóa logistics và vận tải trong hệ thống giao nhận hiện đại

Bối cảnh và đặt vấn đề

Đặt vấn đề

VRPTW không chỉ yêu cầu xác định lộ trình tối ưu cho các phương tiện mà còn phải đảm bảo mỗi khách hàng được phục vụ trong khoảng thời gian quy định.

Đây là bài toán thuộc lớp NP-hard, có độ phức tạp tính toán cao và mang nhiều ý nghĩa trong cả nghiên cứu lý thuyết lẫn ứng dụng thực tiễn.

Lợi ích

Việc giải quyết bài toán VRPTW giúp:

- Giảm tổng quãng đường di chuyển
- Giảm chi phí nhiên liệu
- Tối ưu số lượng phương tiện sử dụng

Mục tiêu nghiên cứu

- Tìm hiểu tổng quan bài toán VRPTW trong lĩnh vực logistics và vận tải.
- Xây dựng mô hình toán học gồm hàm mục tiêu, biến quyết định và các ràng buộc.
- Nghiên cứu các phương pháp giải chính xác như: Branch and Bound, Branch and Cut, Branch and Price, Branch and Cut and Price.
- Thực nghiệm trên bộ dữ liệu Solomon Benchmark.
- Phân tích và đánh giá ưu điểm, hạn chế của các phương pháp giải chính xác.

2.MÔ TẢ BÀI TOÁN

Định nghĩa

Vehicle Routing Problem (VRP) là bài toán tối ưu nhằm tìm lộ trình di chuyển cho một đội xe để phục vụ tập khách hàng với chi phí (hoặc thời gian) nhỏ nhất.

Đây là bài toán mở rộng của Traveling Salesman Problem (TSP) và thuộc lớp NP-hard.

Mở rộng VRP

VRPTW (Vehicle Routing Problem with Time Windows) bổ sung ràng buộc thời gian phục vụ cho mỗi khách hàng.

Đặc trưng

- Bài toán được định nghĩa bởi một đội xe K , một tập khách hàng V và một đồ thị có hướng G . Đội xe được coi là đồng nhất. Đồ thị bao gồm $|V| + 1$ đỉnh, trong đó các khách hàng được đánh số là $1, 2, \dots, n$ và kho bãi (debot) đánh số là 0 .
- Mỗi khách hàng i có nhu cầu q_i và khung thời gian $[a_i, b_i]$
- Xe có sức chứa Q
- Nếu đến sớm, xe phải chờ đến a_i
- Depot có khung thời gian $[a_0, b_0]$

Chi phí

Mỗi cung (i, j) có:

- Chi phí di chuyển c_{ij}
- Thời gian di chuyển t_{ij}

Mục tiêu

- Tìm tập tuyến đường cho đội xe
- Mỗi khách hàng được phục vụ đúng 1 lần
- Thỏa mãn ràng buộc sức chứa và thời gian
- Tối thiểu hóa tổng chi phí di chuyển

Phương pháp giải chính xác

Các phương pháp

- Branch and Bound
- Branch and Cut
- Branch and Price
- Branch and Cut and Price

Ý tưởng chung

Kết hợp phân nhánh (Branching) và kỹ thuật cắt tỉa (Cutting / Pricing) để tìm nghiệm tối ưu toàn cục.

Đặc điểm

Bộ dữ liệu chuẩn do Solomon (1987) đề xuất, dùng để đánh giá VRPTW.

- Depot + tập khách hàng
- Tọa độ, nhu cầu, thời gian phục vụ
- Khung thời gian phục vụ (time window)
- Số xe và sức chứa

Phân loại

- C: khách hàng theo cụm
- R: phân bố ngẫu nhiên
- RC: kết hợp

3.MÔ HÌNH HÓA BÀI TOÁN

Tập hợp và tham số

Tập hợp

- $V = \{0, 1, \dots, n\}$: depot (0) và các khách hàng
- $K = \{1, 2, \dots, m\}$: tập xe

Tham số

- q_i : nhu cầu khách hàng, Q : sức chứa xe
- c_{ij}, t_{ij} : chi phí và thời gian di chuyển
- $[a_i, b_i]$: khung thời gian phục vụ
- s_i : thời gian phục vụ tại điểm i

Biến quyết định và hàm mục tiêu

Biến quyết định

- $x_{ij} \in \{0, 1\}$: đi từ i đến j
- $s_i \geq 0$: thời gian bắt đầu phục vụ
- u_i : tải trọng tại mỗi điểm

Hàm mục tiêu

$$\min Z = \sum_{i \in V} \sum_{\substack{j \in V \\ j \neq i}} c_{ij} x_{ij} \quad (1)$$

Các ràng buộc chính

Ràng buộc tuyến đường

- Mỗi khách hàng được phục vụ đúng 1 lần
- Bảo toàn luồng
- Số xe không vượt quá m

Ràng buộc thời gian và tải trọng

- $a_i \leq s_i \leq b_i$
- Ràng buộc thứ tự thời gian (Big-M)
- Giới hạn tải trọng Q

Ý nghĩa mô hình

- Đảm bảo tính khả thi của lộ trình
- Tối ưu chi phí vận chuyển
- Đáp ứng ràng buộc thời gian và năng lực xe

Mô hình

Hàm mục tiêu:

$$\min Z = \sum_{i \in V} \sum_{j \in V, j \neq i} c_{ij} x_{ij} \quad (1)$$

Ràng buộc:

$$\sum_{j \in V, j \neq i} x_{ij} = 1 \quad \forall i \in V \setminus \{0\} \quad (2)$$

$$\sum_{i \in V, i \neq h} x_{ih} = \sum_{j \in V, j \neq h} x_{hj} \quad \forall h \in V \quad (3)$$

$$\sum_{j \in V \setminus \{0\}} x_{0j} \leq m \quad (4)$$

$$a_i \leq s_i \leq b_i \quad \forall i \in V \quad (5)$$

$$s_i + t_{ij} + s_i^{\text{service}} - s_j \leq M(1 - x_{ij}) \quad \forall i, j \in V \quad (6)$$

$$u_i + q_j - u_j \leq Q(1 - x_{ij}) \quad \forall i, j \in V \quad (7)$$

$$q_i \leq u_i \leq Q \quad \forall i \in V \setminus \{0\} \quad (8)$$

$$x_{ij} \in \{0, 1\} \quad \forall i, j \in V \quad (9)$$

4. BRANCH AND BOUND

Khái quát

Branch and Bound (B&B) là phương pháp giải chính xác cho các bài toán tối ưu tổ hợp và quy hoạch nguyên.

Ý tưởng chính:

- Phân chia bài toán thành các bài toán con (*Branching*)
- Tính cận trên/cận dưới (*Bounding*)
- Loại bỏ các nhánh không tiềm năng (*Pruning*)

Đặc điểm

- Bắt đầu từ LP Relaxation
- Nếu nghiệm nguyên $\Rightarrow$ nghiệm tối ưu
- Nếu nghiệm phân số $\Rightarrow$ tiếp tục phân nhánh

Branching

Nếu nghiệm LP chứa biến phân số, ví dụ:

$$x_{12} = 0.43$$

Tạo hai nhánh:

- $x_{12} = 0$
- $x_{12} = 1$

Bounding

Giải LP Relaxation tại mỗi nút để tính cận dưới (LB).

Pruning

Loại bỏ nhánh nếu:

- Bài toán con vô nghiệm

- 1 Khởi tạo nút gốc và giải LP Relaxation
- 2 Chọn một nút trong cây tìm kiếm
- 3 Tính cận dưới Z_{LP}
- 4 Kiểm tra điều kiện cắt tỉa
- 5 Nếu cần, phân nhánh trên biến phân số
- 6 Đưa các nút con vào hàng đợi
- 7 Lặp lại cho đến khi cây tìm kiếm rỗng

Kết quả

Nghiệm nguyên tốt nhất thu được chính là nghiệm tối ưu của VRPTW.

Pseudocode Branch and Bound

BranchAndBound():

```
best_val = +∞
queue.push(root)

while queue not empty:
    node = queue.pop()

    Z_LP = SolveLP(node)

    if infeasible:
        continue

    if Z_LP >= best_val:
        continue

    if solution integer:
        best_val = Z_LP
        continue

    choose fractional x_ij

    left : x_ij = 0
    right: x_ij = 1

    queue.push(left)
    queue.push(right)

return best_val
```

Độ phức tạp thuật toán

Độ phức tạp

- Số biến nhị phân: n^2
- Số nút tối đa của cây:

$$O(2^{n^2})$$

- Chi phí giải LP tại mỗi nút:

$$O((m \cdot n^2)^{1.5})$$

Độ phức tạp tổng thể

$$T(n) = O\left(2^{n^2} \cdot (m \cdot n^2)^{1.5}\right)$$

Thực tế

Nhờ kỹ thuật cắt tỉa, số nút thực tế thường nhỏ hơn rất nhiều.

5. BRANCH AND CUT

Khái quát

- Kết hợp **Branch-and-Bound** + **Cutting Planes**
- Chủ động thêm các ràng buộc cắt dựa trên khối lượng để thắt chặt LP relaxation
- Mục tiêu: Thắt chặt Lower Bound hơn $\rightarrow$ Tỉa nhánh hiệu quả hơn
- Triển khai trên Google OR-Tools (GLOP) vì không hỗ trợ MIP tự động

Ý tưởng thuật toán

Branch-and-Cut hoạt động dựa trên 4 thành phần chính:

- 1 **Branching**: Phân tách biến phân số $x_{ij}^* \notin \{0, 1\}$ thành $x_{ij} = 0$ và $x_{ij} = 1$
- 2 **Cutting Planes**: Thực hiện Separation để tìm và thêm capacity cuts → subtour elimination
- 3 **Bounding**: Giải LP relaxation để tính Lower Bound Z_{LP} (càng chặt càng tốt)
- 4 **Pruning**: Loại nhánh nếu vô nghiệm, $Z_{LP} \geq UB$, hoặc nghiệm đã nguyên

❶ Giải LP relaxation $\rightarrow$ thu được nghiệm x^* và **Lower Bound** Z_{LP}

❷ **Vòng lặp Cutting Plane (dựa trên khối lượng):**

- Dùng Union-Find tìm tập liên thông S ($\epsilon = 10^{-6}$)
- Tính $k(S)$ là số xe tối thiểu cho tập S
- Tính $\text{flowOut}(S)$ là số xe đang tính cho tập S
- Nếu $\text{flowOut}(S) < k(S) \rightarrow$ thêm ràng buộc cắt:

$$\sum_{i \in S} \sum_{j \notin S} x_{ij} \geq k(S)$$

❸ Cập nhật $\text{gap} = \frac{UB - Z_{LP}}{UB}$. Nếu $\text{gap} < 5\% \rightarrow$ break.

❹ Nếu $x_{ij} \in \{0, 1\} \quad \forall i, j \in V$:

- Cập nhật **Upper Bound** $UB \leftarrow Z_{LP}$
- Lưu lại nghiệm tốt nhất

❺ Ngược lại:

- Chọn biến x_{ij} gần 1 nhất
- Phân nhánh thành hai trường hợp: $x_{ij} = 0$ và $x_{ij} = 1$

Ví dụ minh họa Cutting Plane

- Tập S có tổng nhu cầu $d(S) = 45$, dung lượng $Q = 20$
- $\Rightarrow k(S) = \lceil d(S)/Q \rceil = \lceil 45/20 \rceil = 3$
- $\Rightarrow$ Phải có ít nhất 3 xe rời khỏi tập S
- VD Nếu $\text{flowOut}(S) = 2.85 < k(S) = 3 \rightarrow$ Thêm ràng buộc cắt

Lợi ích: Thắt chặt LP relaxation, tăng Lower Bound nhanh chóng

Pseudocode - Branch-and-Cut

```
1 BranchAndCut():
2     root ← LP Relaxation()
3     manualBranchAndCut(root)
4
5 manualBranchAndCut(node):
6     Solve LP relaxation → Z_LP           // Lower Bound
7     // Cutting Plane Phase
8     while gap > 0.05:
9         S ← findViolatedSubtour()        // Union-Find
10        if S == null: break
11        k_S ← ceil( sum(d_i for i in S) / Q )
12        Add Capacity Cut:
13            x_ij ≥ k_S    ( i ∈ S, j ∉ S )
14        Re-solve LP
15        Update Z_LP and gap
16
17    if infeasible or Z_LP ≥ UB: return
18    if solution is integer:
19        Update UB and best solution
20        return
21
22    (i,j) ← variable closest to 1
23    Branch on x_ij = 0 and x_ij = 1
```

- **Lý thuyết:**

$$T(n) = O\left(2^{n^2} \cdot T_{LP}(m, n^2)\right)$$

- **Thực tế:** Giảm mạnh số nút nhờ:

- Cutting Planes động
- Separation heuristic (Union-Find + HashSet)
- Upper Bound tốt từ Greedy
- Pruning sớm

→ **Hiệu quả cao hơn Branch-and-Bound thuần túy**

6.BRANCH AND PRICE

Branch and Price (Nhánh và Giá) kết hợp hai kỹ thuật:

- **Branch and Bound** – tìm kiếm theo cây, cắt tỉa nhánh không tối ưu.
- **Column Generation** – sinh cột (tuyến đường) có ích theo từng bước.

Trong VRPTW, mỗi *biến* = một tuyến đường khả thi thỏa mãn **sức chứa** và **cửa sổ thời gian**.

⇒ Số tuyến đường tăng theo hàm mũ ⇒ Column Generation chỉ sinh tuyến đường *thực sự có ích*.

Ý tưởng thuật toán – Quy trình tổng quát

Tại mỗi **nút** của cây B&B, thay vì giải LP đơn giản, ta giải *Restricted Master Problem* (RMP) qua Column Generation:

- ❶ **Khởi tạo:** Mỗi khách hàng i được phục vụ bởi tuyến $[0, i, N-1]$.
- ❷ **Column Generation tại mỗi nút:** Giải RMP, sinh thêm cột có *reduced cost* âm cho đến khi không còn cột nào.
- ❸ **Phân nhánh theo cạnh (i, j) :**
 - Nhánh 0: Cấm (i, j) – đặt $\text{dist}[i][j] = +\infty$.
 - Nhánh 1: Bắt buộc (i, j) – cấm mọi cạnh ra/vào trừ (i, j) .
- ❹ **Cắt tỉa:** Loại nút nếu vô nghiệm, cận dưới $>$ cận trên, hoặc nghiệm đã nguyên.

Column Generation – Master Problem (RMP)

Bài toán Set Covering với tập cột tăng dần:

$$\min \sum_{r \in R} c_r y_r \quad \text{s.t.} \quad \sum_{r \in R} a_{ir} y_r \geq 1 \quad \forall i, \quad y_r \geq 0$$

- c_r : chi phí tuyến đường r ; $a_{ir} = 1$ nếu khách $i \in r$.
- Giải bằng **Gurobi** (CONTINUOUS) với *incremental column addition* – không tái tạo mô hình mỗi vòng lặp.

Vòng lặp Column Generation:

- 1 Giải RMP $\Rightarrow$ lấy dual variables π_i .
- 2 Cập nhật $\text{cost}[i][j] = \text{dist}[i][j] - \pi_i$.
- 3 Giải SPPRC tìm tuyến đường có reduced cost âm.
- 4 Thêm tuyến đường mới vào RMP; lặp lại cho đến khi không còn cột mới.

Column Generation – Subproblem: SPPRC

Bài toán con là **SPPRC** (Shortest Path Problem with Resource Constraints): tìm tuyến đường r có *reduced cost* nhỏ nhất:

$$\bar{c}_r = c_r - \sum_i \pi_i \cdot a_{ir}$$

Thuật toán Label Setting (Irnich & Desaulniers):

- Mỗi *label* $L = (v, \text{cost}, \text{ttime}, \text{demand}, \text{visited})$.
- **Mở rộng label** từ $u \rightarrow v$:

$$\text{ttime}' = \max(a_v, \text{ttime} + \text{ttime}[u][v] + s_u), \quad \text{demand}' = \text{demand} + d_v$$

- Label được chấp nhận nếu $\text{ttime}' \leq b_v$ và $\text{demand}' \leq Q$.
- **Dominance**: L_1 loại L_2 nếu L_1 tốt hơn hoặc bằng L_2 trên mọi chiều.
- **Feillet 2004**: đánh dấu sớm các khách hàng không thể thăm tiếp theo để giảm không gian tìm kiếm.

Các label đến depot cuối có $\text{cost} < 0$ được thu thập, truy vết ngược tái tạo tuyến đường.

Sau khi Column Generation hội tụ mà nghiệm còn *phân số*, chọn cạnh phân nhánh:

$$(i^*, j^*) = \arg \max_{0 < x_{ij} < 1} \min(x_{ij}, 1 - x_{ij}) \cdot c_r$$

Nhánh 0 – Cấm (i, j)

- $\text{dist}[i][j] \leftarrow +\infty$

Nhánh 1 – Bắt buộc (i, j)

- Cấm mọi cạnh ra khỏi i (trừ (i, j))
- Cấm mọi cạnh vào j (trừ (i, j))
- Cấm cạnh ngược (j, i)

Cơ chế đệ quy: khi xử lý nút mới, toàn bộ chuỗi ràng buộc từ gốc đến nút hiện tại được tái áp dụng lên ma trận dist .

Chiến lược tìm kiếm: Depth-First Search (DFS).

Cập nhật cận

- **Cận dưới (LB):** $LB = \min(\text{son0.val}, \text{son1.val})$ tại nút cấp cao nhất.
- **Cận trên (UB):** Cập nhật khi tìm được nghiệm nguyên khả thi tốt hơn.

Điều kiện cắt tỉa

- **Gap đủ nhỏ:** $\frac{UB - LB}{UB} < \varepsilon \Rightarrow$ kết thúc thuật toán.
- **Vô nghiệm:** Giá trị CG $> 2 \times \text{maxlength}$ hoặc $< -\varepsilon$.
- **Bound cut:** Giá trị CG tại nút $> UB \Rightarrow$ cắt tỉa nhánh.

- n = số khách hàng, $|R|$ = số cột trong RMP.
- **Column Generation / nút:**
 - Giải LP: $O(n \cdot |R|^{1.5})$ (Simplex).
 - SPPRC: $O(2^n \cdot n^2)$ worst-case (không gian trạng thái *visited*).
- **Cây B&B:** Tối đa $O(2^{n^2})$ nút (số biến nhị phân x_{ij}).
- **Tổng thể (worst case):**

$$T(n) = O(2^{n^2} \cdot 2^n \cdot n^4)$$

- **Thực tế:** Với bộ dữ liệu Solomon (25–100 KH), thuật toán hội tụ nhanh nhờ:
 - Dominance trong SPPRC.
 - Cắt tỉa hiệu quả từ cận dưới CG.
 - Kỹ thuật Feillet 2004 giảm số label.

7.KẾT QUẢ THỰC NGHIỆM

Kết quả so sánh

Method	$N = 5$		$N = 10$		$N = 15$		$N = 100$	
	Obj↓	Time	Obj↓	Time	Obj↓	Time	Obj↓	Time
C101								
B&B	42.4	0.01s	58.2	59.80s	-	-	-	-
B&C	42.4	0.06s	58.2	29.68s	-	-	-	-
B&P	42.4	0.01s	58.2	0.2s	142.1	0.44s	-	-
R101								
B&B	156.3	0.03	269.4	18.92s	-	-	-	-
B&C	156.3	0.07	269.4	11.95s	-	-	-	-
B&P	156.3	0.01	269.4	0.02s	383.7	0.03s	1643.0	151.84s
RC101								
B&B	89.2	0.06	185.9	20.90s	-	-	-	-
B&C	89.2	0.11	185.9	19.85s	-	-	-	-
B&P	89.2	0.01s	185.9	0.11s	228.4	0.26s	-	-

Bảng: Kết quả so sánh các thuật toán với quy mô khách hàng nhỏ và trung bình

8.KẾT LUẬN

Kết luận

Đánh giá phương pháp

- Branch-and-Cut cân bằng tốt giữa chất lượng nghiệm và thời gian tính toán.
- Branch-and-Price phù hợp với các instance lớn nhờ khai thác hiệu quả cấu trúc route.
- Cutting planes giúp cải thiện LP Relaxation và tăng hiệu quả cắt tỉa.

Kết luận thực nghiệm

Các kỹ thuật cải tiến như:

- Cutting planes
- Upper bound từ heuristic
- Pruning sớm

đã giúp tăng đáng kể hiệu suất so với phương pháp cơ bản.